



Pichulman Miguel Angel
Trabajo Práctico II: "SOLID "

Objetivos

- Comprender los principios SOLID.
- Diseñar software flexible y mantenible.
- Reducir acoplamiento y fragilidad.
- Aplicar buenas prácticas de orientación a objetos.
- Mejorar la extensibilidad del sistema.

Contexto

El Hospital Central continúa evolucionando su sistema de gestión de turnos médicos. Ahora el sistema debe enviar notificaciones a pacientes mediante diferentes canales: email, SMS y WhatsApp.

El diseño actual funciona, pero presenta problemas de escalabilidad y mantenimiento. Cada vez que se agrega un nuevo medio de notificación, el código debe modificarse, aumentando el riesgo de errores y deuda técnica.

El objetivo de este trabajo práctico será rediseñar el sistema aplicando correctamente los principios SOLID.

Código Base

Se adjunta en un archivo.

Actividades

Parte A — Diagnóstico del Diseño

- Indicar qué principios SOLID se están violando.
 - SRP: la clase Notificador realiza múltiples tareas.
 - OCP: si mas adelante se desean agregar mas funcionalidades debería modificarse el método enviar.
- Explicar por qué el diseño actual es frágil.
 - Cualquier modificación o corrección en la lógica de un canal se realiza dentro del mismo método donde conviven los demás canales, lo que podría introducir errores involuntarios en Email, whatsapp o sms
- Describir los problemas de mantenibilidad del código.
 - A medida que surjan mas canales, el método enviar acumulara mas "if" haciendo que el flujo de ejecución sea cada vez mas difícil de leer, entender y rastrear. El uso de cadenas de texto literales para determinar el flujo de ejecución es propenso a errores de tipeo en tiempo de ejecución. Hay dificultad para ejecutar pruebas de manera aislada para cada cana, la misma requiere instanciar toda la clase.
- Explicar cómo impacta esto en el costo del cambio.
 - El código esta acoplado y no modularizado, lo que hace que el tiempo de desarrollo aumente cada vez que se solicita una nueva funcionalidad. Cada pequeña adición requiere volver a probar todos los canales existentes para garantizar que no hayan efectos colaterales. La falta de abstracción e interfaces aumenta la deuda técnica, haciendo que el mantenimiento a mediano y largo plazo sea costoso y riesgoso.



Pichulman Miguel Angel

Parte B — Refactor Aplicando SOLID

Rediseñar el sistema aplicando correctamente los principios SOLID.

El nuevo diseño debe incluir:

- Una interfaz Notificacion.
- Implementaciones independientes para cada tipo de notificación.
- Separación clara de responsabilidades.
- Desacoplamiento entre clases.

Ejemplo esperado

```
public interface Notificacion {  
  
    void enviar(String mensaje);  
  
}
```

```
public class NotificadorService {  
    private Notificacion notificacion;  
    public NotificadorService(Notificacion notificacion) {  
        this.notificacion = notificacion;  
    }  
    public void setNotificacion(Notificacion notificacion) {  
        this.notificacion = notificacion;  
    }  
    public void enviarNotificacion(String mensaje) {  
        this.notificacion.enviar(mensaje);  
    }  
}
```

```
public class EmailNotificacion implements Notificacion{  
    @Override  
    public void enviar(String mensaje){  
        System.out.println("Enviando email:" + mensaje);  
    }  
}
```

```
public class SmsNotificacion implements Notificacion {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando sms:" + mensaje);  
    }  
}
```

```
public class WhatsappNotificacion implements Notificacion{  
    @Override  
    public void enviar(String mensaje){  
        System.out.println("Enviando Whatsapp:" + mensaje);  
    }  
}
```



Implementaciones mínimas requeridas

- EmailNotificacion
- SmsNotificacion
- WhatsappNotificacion

Parte C — Extensibilidad

El hospital desea agregar nuevos medios de comunicación.

Incorporar:

- TelegramNotificacion
- PushNotificacion

Condición obligatoria: Las nuevas funcionalidades deben agregarse sin modificar las clases existentes.

```
public class PushNotificacion implements Notificacion {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando Notificación Push: " + mensaje);  
    }  
}
```

```
public class TelegramNotificacion implements Notificacion {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando Telegram: " + mensaje);  
    }  
}
```

Pichulman Miguel Angel

Parte D — Dependency Inversion Principle (DIP)

Crear una clase llamada:

```
public class GestorTurnos
```

Esta clase deberá depender de abstracciones y no de implementaciones concretas.

El sistema debe permitir cambiar fácilmente el mecanismo de notificación.

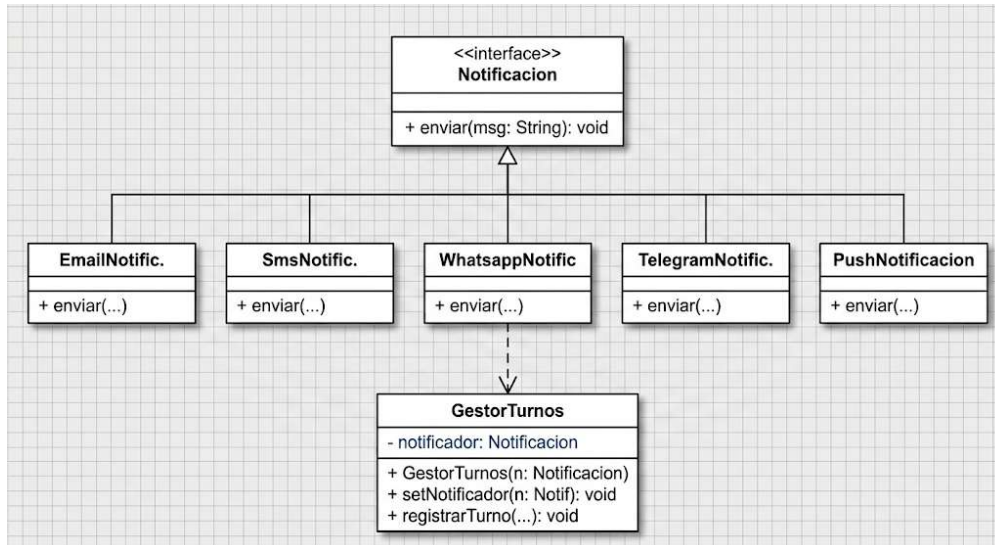
Requisitos Técnicos

- Utilizar Java orientado a objetos.
- Aplicar encapsulamiento correctamente.
- Evitar condicionales innecesarios.
- Utilizar polimorfismo.
- El diseño debe ser extensible y mantenible.

Sugerencia: Se recomienda utilizar composición e interfaces para reducir acoplamiento.

Entregables

- Documento PDF explicando los principios SOLID aplicados.
- Diagrama UML simple del diseño propuesto.



- Código fuente completo en Java.
- Ejemplos de ejecución del sistema.

```
Run tp2 [Main.main()] x
tp2 [Main.main()]: successful At 26/08/2026 1:10 378 ms
New Minor Gradle Version Available

> Task :classes
> Task :Main.main()
Enviando email:Turno confirmado para Juan Perez el día 28/08/2026 10:00 en Cardiología.
Enviando Whatsapp:Turno confirmado para Maria Lopez el día 28/08/2026 11:30 en Dermatología.
Enviando Telegram: Turno confirmado para Carlos Gomez el día 29/08/2026 09:00 en Traumatología.

BUILD SUCCESSFUL in 291ms
2 actionable tasks: 2 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.3.0/userguide/configuration_cache_enabling.html
1:10:26: Execution finished 'Main.main()'.
```